

Network Protocols Reference

Study guide for the netlink sniffer module — protocols, layers, and fields

The OSI model organizes network communication into 7 layers. Each protocol lives at a specific layer. Higher layers depend on lower layers — TCP needs IP, IP needs Ethernet. The netlink sniffer detects protocols in order from highest to lowest to avoid misidentification.

OSI Layer Quick Reference

Layer	Name	Protocols in netlink
7 — Application	Application	HTTP, DNS, DHCP, TLS
6 — Presentation	Presentation	TLS (encryption/encoding)
5 — Session	Session	TLS (session management)
4 — Transport	Transport	TCP, UDP
3 — Network	Network	IP, IPv6, ICMP
2 — Data Link	Data Link	Ethernet, ARP, Dot1Q (VLAN), STP
1 — Physical	Physical	Not captured by Scapy

Netlink Detection Order

Detection order: TLS → HTTP → DNS → DHCP → TCP → UDP → ICMP → IPv6 → ARP → VLAN (Dot1Q) → STP → Other. More specific protocols are checked before generic ones to avoid misidentification. For example DNS is checked before UDP because DNS runs on top of UDP.

Application Layer Protocols

TLS — Transport Layer Security

TLS is the encryption protocol that secures most of the internet. When you see https:// in a browser, TLS is doing the encryption. It replaced SSL (which is now deprecated). TLS operates above TCP and performs a handshake to establish a shared secret before any application data flows. Once the handshake is complete all application data is encrypted — Scapy can see TLS packets exist but cannot read their contents without the session keys.

Common ports: 443 (HTTPS), 465 (SMTPS), 993 (IMAPS), 5986 (WinRM HTTPS)

Field	Description
type	Record type: 20=ChangeCipherSpec, 21=Alert, 22=Handshake, 23=ApplicationData
version	TLS version: 0x0301=TLS1.0, 0x0302=TLS1.1, 0x0303=TLS1.2, 0x0304=TLS1.3
msg	List of TLS message records in this packet (handshake messages, alerts, etc.)
Netlink one-liner:	
[TLS] 192.168.1.86:52341 -> 140.82.112.26:443 (94 bytes)	

HTTP / HTTPRequest / HTTPResponse — HyperText Transfer Protocol

HTTP is the foundation of data communication on the web. It is a request-response protocol — a client sends a request and the server sends a response. HTTP traffic is cleartext (unencrypted) which makes it extremely valuable for credential hunting and traffic analysis. HTTPS is HTTP over TLS. Scapy separates HTTP into HTTPRequest and HTTPResponse layers.

Common ports: 80 (HTTP), 8080 (HTTP-ALT), 8000, 8008

Field	Description
Method	HTTP verb: GET, POST, PUT, DELETE, HEAD, OPTIONS, PATCH
Path	The resource path being requested e.g. /login or /api/v1/users
Http_Version	HTTP version: HTTP/1.0, HTTP/1.1, HTTP/2
Status_Code	Response status: 200=OK, 301=Redirect, 401=Unauthorized, 404=NotFound, 500=Error
Content_Length	Size of the response or request body in bytes
Netlink one-liner:	
[HTTP] 192.168.1.86:52341 -> 140.82.112.26:80 (512 bytes)	

DNS — Domain Name System

DNS is the phone book of the internet — it translates domain names like google.com into IP addresses. Every time you open a browser or app, DNS queries happen first. DNS typically uses UDP for queries under 512 bytes and TCP for larger responses or zone transfers. DNS traffic is cleartext and reveals exactly what domains a host is communicating with — valuable for behavioral analysis and C2 detection.

Common ports: 53/UDP (queries), 53/TCP (zone transfers)

Field	Description
qr	Query or Response flag: 0=Query, 1=Response

qd.qname	The domain being queried e.g. b'google.com.' (bytes — decode with .decode())
an.rdata	The answer — resolved IP address for A records, domain for CNAME, etc.
qdcount	Number of questions in the packet
ancount	Number of answers in the packet
Netlink one-liner: [DNS] 192.168.1.86:52341 -> 8.8.8.8:53 (72 bytes)	

DHCP / BOOTP — Dynamic Host Configuration Protocol

DHCP automatically assigns IP addresses, subnet masks, gateways, and DNS servers to devices when they join a network. It uses a four-step process: DISCOVER, OFFER, REQUEST, ACK. The initial DISCOVER is broadcast before the client has an IP so src=0.0.0.0 and dst=255.255.255.255. DHCP is built on top of BOOTP (Bootstrap Protocol) which is its predecessor. Rogue DHCP servers are a common attack vector for MITM positioning.

Common ports: 67/UDP (server), 68/UDP (client)

Field	Description
options	List of DHCP option tuples: [('message-type', 1), ('server_id', '192.168.1.1'), 'end']
op (BOOTP)	Operation: 1=BOOTREQUEST (client to server), 2=BOOTREPLY (server to client)
htype (BOOTP)	Hardware type: 1=Ethernet
Netlink one-liner: [DHCP] 0.0.0.0:68 -> 255.255.255.255:67 (342 bytes)	

Transport Layer Protocols

TCP — Transmission Control Protocol

TCP is a connection-oriented protocol that guarantees reliable, ordered delivery of data. Before any data flows, TCP performs a three-way handshake (SYN, SYN-ACK, ACK). TCP uses flags to control the connection state and sequence numbers to ensure data arrives in order and without gaps. Most application protocols (HTTP, HTTPS, SSH, SMTP) use TCP because they require reliable delivery.

Field	Description
sport	Source port — the ephemeral port on the sender (typically 1024-65535)
dport	Destination port — the well-known port of the service being reached
flags	Control flags: S=SYN, A=ACK, F=FIN, R=RST, P=PSH, U=URG. Common combos: SA=SYN-ACK, FA=FIN-ACK, RA=RST-ACK
seq	Sequence number — position of this segment in the data stream
ack	Acknowledgment number — next expected byte from the other side
window	Receive window size — how much data the sender can accept
TCP flags you will see most: SYN (new connection), SYN-ACK (connection accepted), RST (connection rejected or reset), FIN (graceful close), PSH (send data immediately).	
Netlink one-liner: [TCP] 192.168.1.86:52341 -> 140.82.112.26:443 (94 bytes)	

UDP — User Datagram Protocol

UDP is a connectionless protocol — it fires packets and does not guarantee delivery, ordering, or error checking. This makes it much faster than TCP with lower overhead. Used for applications where speed matters more than reliability: DNS lookups, video streaming, VoIP, gaming, and DHCP. UDP has no handshake — packets are sent immediately.

Field	Description
sport	Source port — the ephemeral port on the sender
dport	Destination port — the well-known port of the service
len	Length of the UDP header plus data in bytes
chksum	Checksum for error detection (optional in IPv4, mandatory in IPv6)
Netlink one-liner: [UDP] 192.168.1.86:52341 -> 8.8.8.8:53 (72 bytes)	

Network Layer Protocols

IP — Internet Protocol (IPv4)

IP is the core protocol of the internet — it provides addressing and routing so packets can travel across networks from source to destination. Every device on the internet has an IP address. IP itself is connectionless and unreliable — it does not guarantee delivery. That job falls to TCP above it. IPv4 uses 32-bit addresses written as four octets: 192.168.1.1

Field	Description
src	Source IP address — where the packet came from
dst	Destination IP address — where the packet is going
ttl	Time To Live — decremented by 1 at each router hop. Reaches 0 and packet is dropped. OS fingerprinting hint: 64=Linux/Mac, 128=Windows, 255=Network Device
id	Packet identifier — used to reassemble fragmented packets
proto	Protocol number of the layer above: 1=ICMP, 6=TCP, 17=UDP
flags	Fragmentation flags: DF=Don't Fragment, MF=More Fragments
Fields used in netlink one-liner:	
src and dst extracted for all IP-based protocols	

IPv6 — Internet Protocol version 6

IPv6 is the successor to IPv4, using 128-bit addresses to solve the address exhaustion problem. IPv6 addresses are written as eight groups of four hex digits: 2607:f8b0:4002:c10::66. Modern operating systems prefer IPv6 when available — macOS in particular defaults to IPv6 for DNS and many services. This is why you saw IPv6 addresses in the netlink DNS module on Mac.

Field	Description
src	Source IPv6 address
dst	Destination IPv6 address
hlim	Hop Limit — equivalent to TTL in IPv4
nh	Next Header — protocol of the layer above (same values as IPv4 proto field)
tc	Traffic Class — equivalent to IPv4 Type of Service, used for QoS
Netlink one-liner:	
[IPv6] 2600:1701::1 -> 2607:f8b0::1 (94 bytes)	

ICMP — Internet Control Message Protocol

ICMP is used for network diagnostics and error reporting. The ping command uses ICMP echo request (type 8) and echo reply (type 0). Traceroute uses ICMP time exceeded messages. ICMP has no ports — it communicates using type and code fields instead. Many firewalls block ICMP which is why ping sometimes fails even when a host is alive.

Field	Description
type	Message type: 0=Echo Reply, 3=Destination Unreachable, 8=Echo Request, 11=Time Exceeded

code	Sub-type for the message. For type 3: 0=Net Unreachable, 1=Host Unreachable, 3=Port Unreachable
id	Identifier to match requests with replies
seq	Sequence number to match requests with replies
Netlink one-liner:	
[ICMP] 192.168.1.86 -> 192.168.1.254 (28 bytes)	

Data Link Layer Protocols

Ethernet — IEEE 802.3

Ethernet is the most common data link layer protocol — it handles communication between devices on the same local network segment using MAC addresses. Every network interface has a unique 48-bit MAC address burned in by the manufacturer. Ethernet frames wrap all higher-layer packets and are what Scapy sees first when capturing on an interface.

Field	Description
src	Source MAC address — the sender's hardware address e.g. 46:24:7b:e2:cf:ec
dst	Destination MAC address — ff:ff:ff:ff:ff:ff for broadcast
type	EtherType — identifies the layer 3 protocol: 0x0800=IPv4, 0x0806=ARP, 0x86DD=IPv6, 0x8100=VLAN

ARP — Address Resolution Protocol

ARP resolves IP addresses to MAC addresses on a local network. When a device wants to send a packet to an IP it doesn't know the MAC for, it broadcasts an ARP request: who has 192.168.1.1? The owner replies with its MAC. Results are cached in the ARP table. ARP only works on local subnets — it does not cross routers. ARP spoofing poisons these caches to redirect traffic through an attacker.

Field	Description
op	Operation: 1=Request (who-has), 2=Reply (is-at)
hwsrc	Sender MAC address
psrc	Sender IP address
hwdst	Target MAC address (ff:ff:ff:ff:ff:ff in requests)
pdst	Target IP address being queried
ARP spoofing attack: attacker sends gratuitous ARP replies claiming their MAC owns the router's IP. Victims update their ARP cache and send traffic to the attacker instead of the router. The netlink arp_monitor module detects this by flagging IP-MAC mapping changes.	
Netlink one-liner:	
[ARP] 192.168.1.86 (46:24:7b:e2:cf:ec) -> 192.168.1.254 (ff:ff:ff:ff:ff:ff) (42 bytes)	

Dot1Q — IEEE 802.1Q VLAN Tagging

Dot1Q is the standard for VLAN (Virtual LAN) tagging. VLANs logically segment a physical network into multiple isolated networks. A 4-byte tag is inserted into the Ethernet frame between the source MAC and the EtherType field. This tag identifies which VLAN the frame belongs to. You will see Dot1Q frames on networks with managed switches and VLAN configurations — common in enterprise environments and data centers.

Field	Description
prio	Priority Code Point (PCP) — 3-bit field for QoS priority (0-7)
vlan	VLAN ID — 12-bit field identifying the VLAN (1-4094)
type	EtherType of the encapsulated protocol (same as Ethernet type field)

Netlink one-liner:

```
[VLAN] id=10 src=46:24:7b:e2:cf:ec -> dst=2c:c1:f4:f2:c6:50 (64 bytes)
```

STP — Spanning Tree Protocol (IEEE 802.1D)

STP prevents network loops in Ethernet networks with redundant paths. Without STP, a broadcast frame would loop forever between redundant switches causing a broadcast storm that crashes the network. STP elects a root bridge and blocks redundant ports, creating a loop-free logical topology. You will only see STP frames on networks with managed switches. STP frames never contain IP — they are pure layer 2.

Field	Description
proto	Protocol identifier: always 0 for STP
version	STP version: 0=STP, 2=RSTP (Rapid STP), 3=MSTP (Multiple STP)
Netlink one-liner:	
[STP] proto=0 version=0 (35 bytes)	